PATRONES DE DISEÑO

INTRODUCCIÓN

Un patrón de diseño es una solución general y reusable a un problema recurrente dentro de un determinado contexto.

Los patrones de diseño en la orientación a objetos típicamente muestran relaciones e interacciones entre clases y objetos.

Su objetivo es crear un catálogo que captura la experiencia de diseño en un formato que pueda ser usado de forma efectiva.

- **Patrones arquitectónicos**: Expresan una organización estructural fundamental del software. Provee de un conjunto de subsistemas predefinidos especificando sus responsabilidades y relaciones.
- **Patrones de diseño**: Proveen un esquema para refinar los subsistemas o componentes de un sistema software, y las relaciones existentes entre ellos. Resuelven un problema general de diseño dentro de un contexto particular.
- **Idiomas**: Describen cómo implementar aspectos particulares de componentes utilizando las características de un determinado lenguaje.
- **Antipatrones**: Representan soluciones recurrentes a un problema que, definitivamente, tienen consecuencias negativas.

TIPOS DE PATRONES DE DISEÑO

- **Patrones creacionales**: Tratan sobre cómo crear instancias de objetos y sobre cómo hacer los programas más flexibles y generales abstrayendo el proceso de creación de instancias.
- **Patrones estructurales**: Describen cómo las clases y los objetos pueden ser combinados para formar estructuras mayores.
- **Patrones de comportamiento**: Son patrones que tratan de forma más específica con aspectos relacionados con la comunicación entre objetos.

PATRONES FUNDAMENTALES

Principio “Favorece la inmutabilidad”

Las clases deben ser inmutables, a no ser que haya una buena razón para hacerlas mutables.

Ventajas:

- Son objetos simples ya que no pueden cambiar de estado. Para asegurar que las invariantes se cumplen sólo hay que realizar los constructores correctamente.
- Pueden compartirse abiertamente sin temor a modificaciones.
- No requieren sincronización al usarse en varios hilos de ejecución de forma concurrente.
- Sirven de bloques de construcción para objetos más complejos.

Patrón Inmutable

Se encarga de diseñar clases en las cuales toda la información contenida en cada instancia es proporcionada en el momento de la creación y no puede modificarse durante el tiempo de vida del objeto.

@informaticapau

Reglas para hacer una clase inmutable

1. **No incluir métodos que modifiquen el estado del objeto**: Conocidos como métodos de escritura o mutadores.
2. **Asegurarse de que la clase no puede ser extendida**: De esa forma evitaremos que subclases descuidadas o directamente maliciosas comprometan el comportamiento inmutable. La forma más fácil de hacerlo es definiendo la clase como final.
3. **Declarar todos los atributos como finales**: Nos fuerza a darle un valor a dichos atributos en el constructor ya que dicho valor no pueda ser modificado (*blank finals*).
4. **Declarar todos los atributos como privados**: Evitando que los clientes obtengan acceso a objetos mutables y puedan modificar dichos objetos directamente.
5. **Evitar el acceso a componentes internos mutables**: Si la clase tiene atributos que son objetos mutables entonces hay que asegurarse que ningún cliente de la clase pueda obtener referencias a estos objetos (evitando inicializar estos objetos con referencias obtenidas del cliente, haciendo copias defensivas, etc.).

Inconveniente

Una clase inmutable requiere un objeto distinto por cada posible valor. Crear estos objetos puede ser costoso, especialmente si son grandes. El problema de rendimiento se acentúa si se realiza una operación en varios pasos que genera un nuevo objeto en cada paso, descartando finalmente todos los objetos excepto el resultado final.

Solución

Definir clases mutables asociadas a clases inmutables y que puedan utilizarse para realizar las operaciones que resultan costosas en las clases inmutables. Estas clases sí permiten cambios en el estado en aquellas operaciones en las que la clase inmutable simulaba alterar el estado del objeto. También incluyen métodos para crear un objeto mutable a partir de su equivalente inmutable y viceversa.

Patrón Instancia Única

Se utiliza para asegurarnos que una clase tiene sólo una instancia y proporcionar un punto global de acceso a ella.

Instancia Única: Clase que tiene un atributo del tipo de la propia clase definido como estático y privado. El constructor se define privado para evitar que se creen más instancias. Un método de lectura estático nos permite acceder a dicho atributo.

Problemas con el Patrón Instancia Única:

- Ofrecen un punto global de acceso a un servicio. Eso crea una variable global que constituye una dependencia oculta no visible en el código.
- Llevan un estado con ellos que dura tanto como la ejecución del programa. Los estados persistentes son enemigos de las pruebas unitarias porque estas requieren que los tests sean independientes unos de otros. Valores globales y estáticos como son los *singleton* pueden crear dependencias indeseadas en los tests.
- Estos problemas se minimizan si los *singleton* no guardan ningún estado.

PATRONES ADAPTABLES A LOS CAMBIOS

Principio "Encapsula lo que varía"

Identifica los aspectos de tu aplicación que varían y sepáralos de aquellos que permanecen estables.

@informaticapau

No importa lo bien que esté diseñada una aplicación, con el tiempo una aplicación debe crecer y cambiar o morirá.

Separando las partes susceptibles de variar significa que podemos cambiarlas en el futuro sin afectar a otras partes del código.

Patrón Estrategia

Patrón de comportamiento que se utiliza para definir una familia de algoritmos, encapsularlos y hacerlos intercambiables.

Ventajas:

- Permite representar de forma sencilla familias de algoritmos factorizando sus partes comunes en una misma clase padre.
- Podría hacerse realizando subclases del contexto, pero mezclaríamos algoritmo y contexto complicando su comprensión y/o modificación.
- Evita tener que utilizar múltiples sentencias condicionales en el contexto para elegir el algoritmo adecuado.

Inconvenientes:

- Todos los algoritmos tienen que compartir un mismo interfaz, lo que en ocasiones obliga a pasar datos innecesarios (algún algoritmo requiere menos datos) sobrecargando la comunicación entre contexto y algoritmo.
- Si hay muchas alternativas se pueden llegar a crear muchos objetos. Algo que puede solucionarse implementando las estrategias como objetos sin estado que puedan compartirse entre distintos contextos.

Patrón Estado

Patrón de comportamiento que permite a un objeto modificar su conducta al cambiar su estado interno.

Elementos:

- **Contexto**: Mantiene una instancia de un estado concreto y delega en ella el funcionamiento dependiente del estado.
- **Estado**: Clase abstracta que encapsula el comportamiento de los estados del contexto.
- **Estado concreto**: Implementa el comportamiento asociado con un estado del contexto.

Ventajas:

- Localiza y separa el comportamiento específico de cada estado.
- Permite añadir nuevos estados de forma sencilla, simplemente añadiendo nuevas clases.
- Las transiciones entre estados y el comportamiento de los mismos es más claro.
- Los objetos que representan a los estados pueden compartirse, siempre y cuando no incluyan información en variables de instancia.

Inconveniente:

- El código es menos compacto que la solución basada en sentencias condicionales.

@informaticapau

Implementación con sentencias condicionales

Ventajas:

- Es una solución sencilla y compacta (todo el comportamiento se encapsula en una única clase).
- Es la solución preferida si el número de estados es pequeño, el número de métodos afectados por el estado son pocos y ambos son estables (no variarán en el futuro).

Inconvenientes:

- Solución confusa: demasiadas sentencias condicionales complican su lectura, es complejo saber el funcionamiento de un estado concreto.
- Poco adaptable a cambios: cualquier cambio en los estados o métodos obliga a cambiar la clase incumpliendo principios como el Abierto-Cerrado.
- Poco cohesiva: La clase tiene demasiadas responsabilidades entremezcladas.

DISEÑOS DÉBILMENTE ACOPLADOS

Principio de "Bajo Acoplamiento"

Busca diseños débilmente acoplados entre objetos que interactúan.

Acoplamiento

Medida de la dependencia que tiene una determinada clase de otras clases del sistema.

Un código con bajo acoplamiento es un código más robusto y adaptable, ya que los cambios en una clase no se propagan al resto de clases del sistema, sólo a unas pocas clases cercanas.

¿Cómo reducir el acoplamiento?

Haciendo que la clase trabaje sobre objetos abstractos y no concretos, haciéndolos menos dependientes de una implementación concreta ⇒ Inversión de la Dependencia.

Conectando a las clases con el menor número posible de otras clases, usando clases que juegan el rol de intermediarias entre otras clases del sistema ⇒ Principio de Mínimo Conocimiento.

Acoplamiento y Cohesión

El objetivo es crear objetos cohesivos (centrados en resolver una única responsabilidad) con bajo acoplamiento (dependientes del menor número posible de clases).

Acoplamiento y clases inestables

El acoplamiento alto es un problema cuando alguna de las clases es inestable y cambia frecuentemente, ya que eso provoca cambios en los elementos dependientes. Si las clases acopladas son estables, desacoplarlas lo único que hace es añadir complejidad al sistema.

Patrón Observador

Permite definir una dependencia "uno a muchos" entre objetos de tal forma que, cuando el objeto cambie de estado, todos sus objetos dependientes sean notificados y actualizados automáticamente.

El patrón desacopla al objeto observado de sus observadores, ya que el primero desconoce la clase a la que pertenecen los últimos, sólo sabe que implementan un determinado interfaz.

Se pueden añadir observadores en cualquier momento y no es necesario modificar el sujeto observado al añadir nuevos tipos de observadores.

Permite que los observadores y observados varíen de forma independiente.

@informaticapau

Elementos:

- **Sujeto**: Clase abstracta que está siendo observada y que proporciona la interfaz para mantener y notificar a los observadores.
- **Sujeto concreto**: Mantiene el estado del sujeto y se encarga de notificar a los observadores al cambiar su estado.
- **Observador**: Clase abstracta que define un método "actualizar" utilizado por el sujeto para notificar cambios en su estado.
- **Observador concreto**: Mantiene una referencia al sujeto concreto e implementa "actualizar".

Ventajas:

- Al desacoplar los observadores de la clase observable, esta última no tiene por qué conocer la clase concreta a la que pertenecen sus observadores, simplemente se limita a notificar los cambios a través de un interfaz conocido.
- La responsabilidad del objeto observado se acaba con la notificación, son los observadores los que tienen que decidir qué acción llevar a cabo después de la notificación.

Inconvenientes:

- Los observadores no tienen contacto entre sí, esto puede dar lugar a situaciones difíciles de controlar cuando varios observadores responden simultáneamente ante un cambio.
- En el peor de los casos puede dar lugar a una cascada de actualizaciones.
- La comunicación entre observado y observadores suele ser simple, lo que fuerza a los observadores a deducir qué ha cambiado en el observado.

Modelo pull

- Los observadores "tiran" del sujeto.
- El sujeto manda a los observadores una mínima notificación indicando que su estado ha cambiado.
- Es responsabilidad de los observadores descubrir qué ha cambiado y actuar en consecuencia.

Ventaja: El observado desconoce en general las necesidades de los observadores.

Inconveniente: Puede ser ineficiente, ya que las clases observadoras tienen la sobrecarga de descubrir qué ha cambiado sin la ayuda del sujeto.

Modelo push

- El sujeto "empuja" a los observadores.
- El sujeto manda a los observadores información detallada acerca de lo que ha cambiado.
- Implica que el sujeto tiene un cierto conocimiento sobre las necesidades de los observadores.

Ventaja: La comunicación puede ser más eficiente ya que no se fuerza a descubrir a los observadores qué es lo que ha cambiado.

Inconveniente: Sujeto y observadores no son tan independientes, el sujeto puede hacer asunciones sobre sus observadores que no siempre son ciertas.

@informaticapau

Patrón Adaptador

Convierte el interfaz de una clase en otro interfaz que es el esperado por los clientes. De esta forma permite que cooperen clases con interfaces incompatibles.

El patrón asume que no se pueden hacer cambios ni en el interfaz que esperan los clientes ni en el interfaz que provee la clase servidora. Puede ser debido a que es un API externo, son clases de las cuales no disponemos el código fuente o simplemente no se quiere modificar las clases para un determinado uso concreto y puntual.

Elementos:

- **Cliente**: Colabora con otros objetos usando el interfaz Objetivo.
- **Objetivo**: Interfaz que utiliza el cliente.
- **Adaptado**: Define una interfaz existente que necesita ser adaptada.
- **Adaptador**: Implementa el interfaz Objetivo.

Herencia Múltiple (adaptador de clases)

- Sólo permite adaptar a una clase, no a sus subclases.
- Permite que el adaptador sobrescriba parte de la conducta del adaptado (ya que es una subclase).
- Nos evitamos la indirección que se produce en la composición.

Composición (adaptador de objetos)

- Permite que un único adaptador trabaje sobre muchos adaptados (una clase y sus subclases) añadiendo funcionalidades a todos si lo considera necesario.
- Introduce un nivel más de indirección.
- Complica la sobrescritura de la conducta del adaptado (se necesita crear una subclase del adaptado y que el adaptador se refiera a esta subclase).

Principio de Mínimo Conocimiento (Ley de Deméter)

Un objeto dado debe saber lo menos posible acerca de la estructura o propiedades de otros objetos (ocultación de la información).

Dentro de un método de un determinado objeto sólo se pueden mandar mensajes a:

- El propio objeto.
- Un parámetro del método.
- Un atributo del propio objeto.
- Un elemento de una colección que es un atributo del propio objeto.
- Un objeto creado dentro del método.

Principio “Tell, Don’t Ask”

El código procedural pregunta información (*ask*) y luego toma decisiones. El código orientado a objetos le dice (*tell*) a los objetos que hagan cosas.

Preguntarle a un objeto por su estado y luego llamar a métodos sobre ese objeto basándose en decisiones tomadas fuera de ese objeto, significa que ese objeto es una ***leaky abstraction*** (abstracción con fugas).

Parte del comportamiento del objeto se sitúa fuera del mismo y su estructura interna se expone (puede que innecesariamente) al mundo exterior. Es mejor decirle (tell) a los objetos lo que quieres que hagan.

@informaticapau

Ventajas:

- Anima a combinar datos y comportamiento en una misma abstracción.
- Los elementos que estén estrechamente acoplados deben estar en el mismo componente.
- Lleva a diseños desacoplados de forma similar a como lo hace la Ley de Deméter. Si tu código ya sigue la Ley de Deméter, será más fácil aplicar el principio Tell Don't Ask.
- Se centra en el paso de mensajes entre objetos, enviar instrucciones a un objeto es mejor que consultarlo para realizar acciones.

Inconvenientes:

- Puede alentar a las personas a convertirse en erradicadoras de *getters*, tratando de deshacerse de todos los métodos de consulta, pero hay momentos en que los objetos colaboran efectivamente proporcionando información.
- Puede entrar en conflicto con el Principio de Responsabilidad Única y los objetos pueden crecer y volverse demasiado grandes si no se tiene cuidado.

Patrón Fachada

Provee de un interfaz unificado para un conjunto de clases en un subsistema. La fachada actuaría como un interfaz de alto nivel que haría que el subsistema sea más fácil de utilizar.

- **Fachada**: Conoce qué clases son las responsables de una determinada petición y delega dicha petición sobre esas clases.
- **Clases del subsistema**: Implementan las funcionalidades del subsistema y reciben las peticiones desde la fachada, aunque desconocen su existencia.

Ventajas:

- El subsistema resulta más fácil de utilizar por parte de sus clientes.
- Bajo acoplamiento, la modificación del subsistema no afecta a los clientes que lo utilizan.
- Los clientes pueden seguir usando las clases del subsistema obviando la fachada si consideran que no corresponde con sus necesidades.

Fachada vs. Adaptador

- Se diferencian más en la intención que en su propia estructura.
- El patrón adaptador trata de alterar un interfaz para que coincida con lo que el cliente se espera.
- El patrón fachada trata de ofrecer un interfaz simplificado sobre un subsistema complejo.

PATRONES Y COLECCIONES DE OBJETOS

Patrón Composición

Es un patrón estructural que se utiliza para componer objetos en estructuras de árbol que representan jerarquías todo-parte.

- Este patrón permite tratar uniformemente a los objetos y a las composiciones.
- La clave de este patrón es una clase abstracta que representa al mismo tiempo a los elementos primitivos y a sus contenedores.
- Es un buen ejemplo de utilización conjunta de herencia y composición.

@informaticapau

Ventajas:

- Permite tratar a los objetos básicos y a los objetos compuestos de manera uniforme, ignorando las diferencias entre composiciones de objetos y objetos individuales.
- Facilita la introducción de nuevos componentes.

Inconvenientes:

- Si queremos restringir qué componentes concretos pueden ir en una determinada composición es necesario añadir comprobaciones en tiempo de ejecución.

¿Qué clase debería declarar las operaciones para insertar y borrar hijos en la jerarquía de la composición?

La decisión implica un equilibrio entre seguridad y transparencia.

- Definiendo la gestión de hijos en la clase Composición:
    - Seguridad: cualquier intento de añadir o eliminar objetos de las hojas será detectado en tiempo de compilación en lenguajes tipados estáticamente.
    - Pérdida de transparencia: porque las hojas y las composiciones tienen interfaces diferentes.
- Definiendo la gestión de hijos en la raíz de la jerarquía de clases.
    - Transparencia: porque puedes tratar a todos los componentes de forma uniforme.
    - Coste en seguridad: porque los clientes pueden tratar de hacer cosas sin sentido como añadir o eliminar objetos de hojas.

Patrón Iterador

Proporciona un modo de acceder secuencialmente a los elementos de un objeto agregado (colección de objetos) sin exponer su representación interna.

Elementos:

- **Agregado**: Define una interfaz para crear un objeto Iterador.
- **Agregado concreto**: Representa al objeto agregado que mantiene una colección de objetos y que puede crear iteradores.
- **Iterador**: Define una interfaz para recorrer los elementos y acceder a ellos.
- **Iterador concreto**: Implementa el interfaz Iterador y permite el recorrido del objeto agregado manteniendo la posición actual del mismo.

Ventajas:

- Se puede acceder al contenido del objeto agregado sin exponer su representación interna.
- Permite implementar varios recorridos distintos sobre objetos agregados.
- Proporciona una interfaz uniforme para recorrer diferentes estructuras agregadas.

Inconvenientes:

- La modificación del agregado mientras se está recorriendo con un iterador es una operación peligrosa.
- Una solución sencilla es copiar el agregado y recorrer la copia, pero puede resultar muy costosa.
- Una solución compleja es diseñar iteradores “robustos” que se adapten a estas modificaciones.

@informaticapau

OTROS PRINCIPIOS Y PATRONES

Método Factoría

Define un interfaz para crear un objeto, pero permitiendo que las subclases sean las que decidan qué objeto hay que crear.

Elementos:

- **Producto**: Define el interfaz de los objetos creados por el método factoría.
- **Producto concreto**: Implementa el interfaz de los productos.
- **Creador**: Declara el método factoría, que devuelve un objeto de tipo Producto.
- **Creador concreto**: Sobrescribe el método factoría para devolver una instancia de un ProductoConcreto.

Posibles implementaciones:

- Factoría como método abstracto: El método factoría se declara abstracto en la superclase y debe ser redefinido por las subclases (no hay implementación por defecto).
- Factoría como método concreto: El método factoría no es abstracto (incluye una implementación por defecto).
    - Habitualmente la superclase tampoco es abstracta.
    - No es necesario utilizar las subclases si no se quiere, pero se abre la posibilidad a definir una subclase especializada.
    - Esta versión del método factoría obliga a tener dos jerarquías paralelas, una para los productos y otra para las factorías. Esto puede evitarse con una versión parametrizada de la factoría en la cual sólo existe una clase factoría que crea un producto u otro según un parámetro que se le pasa.
- Factoría parametrizada: Los objetos se crean según un parámetro que se le pasa a la factoría.
    - Se elimina la necesidad de las subclases, aunque el proceso de creación es menos abstracto (a la factoría le indicamos lo que tiene que crear).
    - Para evitar tener que crear una nueva entrada cada vez que se crea un nuevo producto (incumpliendo el principio abierto-cerrado) podemos usar otras estrategias para crear la factoría parametrizada:
        - Reflexión: Usa la información de tipos en tiempo de ejecución para crear objetos a partir del nombre.
        - Genericidad: La factoría usa la genericidad para crear objetos a partir de una referencia a su clase.

Patrón Constructor (*builder*)

Separa la construcción de un objeto complejo de su representación, de forma que un mismo proceso de construcción pueda crear diferentes representaciones.

Elementos:

- **Director**: Construye un objeto usando el interfaz del Constructor.
- **Constructor**: Especifica una interfaz abstracta para crear las partes de un objeto.
- **Constructor concreto**: Implementa el interfaz del constructor para crear un objeto determinado.
- **Producto**: Representa el objeto en construcción.

@informaticapau

Antipatrón Constructor telescópico

Un constructor telescópico es aquel constructor que tiene una versión con los parámetros requeridos, luego otra versión con los parámetros requeridos y un parámetro opcional, luego otra versión con los parámetros requeridos y dos parámetros opcionales, etc.

Problemas:

- El código funciona, pero es complicado escribir código que lo use que sea sencillo y fácil de leer.
- Largas secuencias de parámetros con el mismo tipo pueden causar fallos sutiles. Si accidentalmente cambiamos el orden de dos parámetros el compilador no protestará, pero el código funcionará incorrectamente.

Principio de Hollywood

Consiste en desacoplar elementos de alto nivel y de bajo nivel que pueden trabajar juntos.

El elemento de alto nivel mantiene el control y es el encargado de llamar a los elementos de bajo nivel cuando lo considera necesario.

Los elementos de bajo nivel nunca llaman a los de alto nivel directamente.

Es una forma de inversión del control como también lo es la inyección de dependencias.

Método Plantilla

Define el esqueleto de un algoritmo en una operación, pero difiriendo alguno de los pasos a las subclases.

Las subclases pueden cambiar ciertos aspectos del algoritmo, sin cambiar su estructura general. Debe su nombre a que utilizarlo es como cubrir una plantilla o un formulario.

Patrón Método Plantilla vs. Patrón Estrategia

Similitudes:

- Se utilizan para implementar distintos tipos de algoritmos similares.

Diferencias:

- El patrón estrategia utiliza la delegación (el contexto delega en la estrategia) para escoger entre las distintas variantes de un algoritmo (los algoritmos persiguen el mismo objetivo, pero pueden ser completamente diferentes).
- El patrón método plantilla utiliza la herencia para variar determinadas partes de un algoritmo dado (no se puede variar el esquema general del algoritmo).

Principio "*Don't Repeat Yourself*" (DRY)

Cada pieza de conocimiento debe tener una representación única, inequívoca y autorizada dentro de un sistema.

Añadir código adicional e innecesario al código base aumenta la cantidad de trabajo necesario para ampliar y mantener el software en el futuro.

Tanto si la duplicación proviene de "Programación Corta-Pega" o de una mala comprensión de cómo aplicar la abstracción, disminuye la calidad del código.

Las violaciones del DRY son referidas típicamente como soluciones WET: "*write everything twice*", "*we enjoy typing*" o "*waste everyone's time*".

@informaticapau

Principio "*Keep It Simple, Stupid*" (KISS)

La simplicidad debe ser mantenida como un objetivo clave del diseño, y cualquier complejidad innecesaria debe ser evitada.

Principio "*You Aren't Gonna Need It*" (YAGNI)

Implementar los cambios cuando realmente se necesiten, no cuando sólo se prevea que se necesitarán.

Sobre-Ingeniería (*Over-Engineering*)

Hacer el código más flexible o sofisticado de lo necesario. Se hace para anticipar futuras necesidades, pero predecir el futuro es una tarea arriesgada. Si las predicciones no son correctas perdemos un tiempo valioso en hacer código innecesario. Siguiendo la norma "no arregles lo que no está roto" este código nunca será eliminado, lo que podría complicar futuras modificaciones no previstas.

Infra-Ingeniería (*Under-Engineering*)

Se refiere a código pobremente diseñado y es problema más común que afecta al diseño. Un diseño incorrecto que funcione correctamente nunca será modificado aplicando la norma "no arregles lo que no está roto". A medida que el proyecto va avanzando y se añade más código, más complicado se hace poder corregir el código pobremente diseñado. Al final puede llegarse a la necesidad de realizar una completa reescritura para adaptarse a las nuevas necesidades.

@informaticapau